君正®

T23 Sensor 添加和调试说明

Date: Jun.2023

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

君正®
T23 Sensor 添加和调试说明
Copyright © Ingenic Semiconductor Co. Ltd 2019. All rights reserved.

Release history

Date	Revision	Change
Jun.2023	1.0	First release

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

北京君正集成电路股份有限公司

地址: 北京市海淀区西北旺东路 10 号院东区 14 号楼君正大厦
电话:(86-10)56345000
传真:(86-10)56345001
Http: //www.ingenic.com

目录

1 Sensor 移植基本流程 1

2 Sensor 移植 2

2.1 移植 Sensor 需要文件概述： 2
2.2 移植 ISVP 平台 Sensor 驱动至 Zeratul 平台 2
2.2.1 修改 sensor 驱动 2
2.2.2 编译 sensor 驱动 6
2.3 修改编译脚本 6
2.4 添加 Sensor 所需 TAG 文件 7

3 sensor 问题调试说明 10

2.1 sensor 常见 FAQ 10
Q：ISVP 平台和 Zeratul 平台使用 Sensor 区别 10
Q：运行 sample 程序发现 err:video drop、chip found @0x**（i2c0）is not an ** chip 等错误。 10
Q：运行 sample 程序发现：IMP_Encoder_PollingStream(0) timeout。但是没有 err:video drop 10
Q：运行 sample 程序发现：主码流帧率减半 11
Q: 图像显示不正常 11
Q: 图像显示偏绿 11
Q: 图像显示花屏 11
Q：图像条纹 11
Q: 如何修改 Sensor 默认帧率 12
Q: 关于设置帧率接口，帧率如何定义？Sensor FPS 和 Encoder FPS 有什么区别或联系？ 12
2.2 sensor 不出图像排查 13
2.2.1、Sensor 不出图像硬件问题排查 13
2.2.2 Sensor 不出图像软件问题排查 13

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

1 Sensor 移植基本流程

Step 1、添加<sensor>.hex

Source 环境选择一款目前支持的 sensor，然后手动编译内核
1.2 小节 移植 sensor 驱动源码
1.3 小节 编译 sensor 驱动
1.4 添加脚本内容
重新 source 环境选择添加的 sensor
手动编译内核看是否能够编译通过。若不通过，重新从第一步开始。

Step 2、添加 tag 内容

参考 1.5 小节添加 tag 所需文件
重新 source 环境选择用户自己添加的<sensor>
运行脚本 build/build_camera_tag.sh 看是否编译通过。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

2 Sensor 移植

2.1 移植 Sensor 需要文件概述：

Zeratul 平台的 sensor 移植过程就是制作这些文件的过程。

以 gc2053 为例：

1、sensor 驱动和编译驱动的 Makefile

```
/os/drivers/sensor-t31/zeratul/gc2053$ ls -hl

月  20 16:12 gc2053.c
月  28 17:36 Makefile
```

2、tag 所需文件部分：

```
/tools/make_tag$ ls -hl

16:43 iq_crc
16:43 riscv_fw
16:43 sensor_calibration_bin
16:43 sensor_init_setting
16:43 sensor_start_ae_table
15:41 tag_generator
```

libz_gc2053.c　起始 AE 等内容的定义

gc2053_t23_init.c　初始化 sensor 寄存器列表

riscv_fw_gc2053.bin　riscv 固件

gc2053_t23.bin　sensor 效果文件

2.2 移植 ISVP 平台 Sensor 驱动至 Zeratul 平台

2.2.1 修改 sensor 驱动

这里介绍了 Sensor 驱动移植方法，并且选取部分内容截图，因为不同 Sensor 略有差别，具体移植内容要看提供的驱动源码。

以 GC2053 为例，在 os/drivers/sencor-t23/isvp 路径下找到 gc2053 的 isvp 驱动，对其进行以下修改：

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

添加宏 SENSOR_WITHOUT_INIT

宏 SENSOR_WITHOUT_INIT 对 Sensor 源码的部分进行了选择编译，在调试 Sensor 时候可以注释掉。如图所示，宏 SENSOR_WITHOUT_INIT 具体放置的位置可以参照 Zeratul 下的其他 Sensor 驱动源码。

```
#define GC2053_CHIP_ID_H      (0x20)
#define GC2053_CHIP_ID_L      (0x53)
#define GC2053_REG_END        0xff
#define GC2053_REG_DELAY      0x00
#define GC2053_SUPPORT_30FPS_SCLK (74250000)
#define GC2053_SUPPORT_15FPS_SCLK (48000000)
#define SENSOR_OUTPUT_MAX_FPS 30
#define SENSOR_OUTPUT_MIN_FPS 5
#define SENSOR_VERSION  "H20190820"

//#define SENSOR_RISCV_HF_MODE
#define SENSOR_WITHOUT_INIT
```

```
static int gc2053_init(struct tx_isp_subdev *sd, int enable)
{
    struct tx_isp_sensor *sensor = sd_to_sensor_device(sd);
    int ret = 0;

    if(!enable)
        return ISP_SUCCESS;

    sensor->video.mbus.width = wsize->width;
    sensor->video.mbus.height = wsize->height;
    sensor->video.mbus.code = wsize->mbus_code;
    sensor->video.mbus.field = V4L2_FIELD_NONE;
    sensor->video.mbus.colorspace = wsize->colorspace;
    sensor->video.fps = wsize->fps;

#ifdef SENSOR_RISCV_HF_MODE
    ret = gc2053_write_array(sd, wsize->regs);
    if (ret)
        return ret;
#endif

#ifndef SENSOR_WITHOUT_INIT          kwang, 3年前 · 添加Sensor说
    ret = gc2053_write_array(sd, wsize->regs);
    if (ret)
        return ret;
#endif
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
static int gc2053_g_chip_ident(struct tx_isp_subdev *sd,
                struct tx_isp_chip_ident *chip)
{
    struct i2c_client *client = tx_isp_get_subdevdata(sd);
    unsigned int ident = 0;
    int ret = ISP_SUCCESS;

#ifndef SENSOR_WITHOUT_INIT        kwang, 3年前 · 添加Sensor说明文档参考资料
    if(reset_gpio != -1){
        ret = private_gpio_request(reset_gpio,"gc2053_reset");
        if(!ret){ ···
        }else{
            ISP_ERROR("gpio requrest fail %d\n",reset_gpio);
        }
    }
    if(pwdn_gpio != -1){
        ret = private_gpio_request(pwdn_gpio,"gc2053_pwdn");
        if(!ret){ ···
        }else{
            ISP_ERROR("gpio requrest fail %d\n",pwdn_gpio);
        }
    }
    ret = gc2053_detect(sd, &ident);
    if (ret) { ···
    }
#else
    ident = 0x2053;
#endif
    ISP_WARNING("gc2053 chip found @ 0x%02x (%s)\n", client->addr, client->adapt
    if(chip){ ···
    }
    return 0;
}
```

```
static int gc2053_probe(struct i2c_client *client, const struct i2c_device_id *id)
{
    struct tx_isp_subdev *sd;
    struct tx_isp_video_in *video;
    struct tx_isp_sensor *sensor;
    int ret;
    unsigned int again = 0;

    sensor = (struct tx_isp_sensor *)kzalloc(sizeof(*sensor), GFP_KERNEL);
    if(!sensor){ ···
    }
    memset(sensor, 0 ,sizeof(*sensor));

    sensor->mclk = clk_get(NULL, "cgu_cim");
    if (IS_ERR(sensor->mclk)) { ···
    }

#ifndef SENSOR_WITHOUT_INIT        kwang, 3年前 · 添加Sensor说明文档参考
    private_clk_set_rate(sensor->mclk, 24000000);
    private_clk_enable(sensor->mclk);
#endif
```

函数名称修改：将左边 **ISVP** 的命名改到右边的样子

Isvp

```
static __init int init_gc2053(void)
{
    int ret = 0;
    ret = private_driver_get_interface();
    if(ret){
        ISP_ERROR("Failed to init gc2053 dirver.\n");
        return -1;
    }
    return private_i2c_add_driver(&gc2053_driver);
}

static __exit void exit_gc2053(void)
{
    private_i2c_del_driver(&gc2053_driver);
}
```

Zeratul

```
int init_sensor(void)
{
    int ret = 0;
    ret = private_driver_get_interface();
    if(ret){
        printk("Failed to init gc2035 dirver.\n");
        return -1;
    }

    if(data_type == TX_SENSOR_DATA_TYPE_WDR_DOL) {
        fast_wdr_mode = 1;
    } else {
        fast_wdr_mode = 0;
    }

    return private_i2c_add_driver(&gc2053_driver);
}

void exit_sensor(void)
{
    i2c_del_driver(&gc2053_driver);
}
```

其他添加部分：

```
extern int riscv_is_pass;

static int reset_gpio = GPIO_PA(18);
//module_param(reset_gpio, int, S_IRUGO);
//MODULE_PARM_DESC(reset_gpio, "Reset GPIO NUM");

static int pwdn_gpio = -1;
//module_param(pwdn_gpio, int, S_IRUGO);
//MODULE_PARM_DESC(pwdn_gpio, "Power down GPIO NUM");

static int sensor_gpio_func = DVP_PA_LOW_10BIT;
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
static int gc2053_init(struct tx_isp_subdev *sd, int enable)
{
    struct tx_isp_sensor *sensor = sd_to_sensor_device(sd);
    int ret = 0;

    if(!enable)
        return ISP_SUCCESS;

    sensor->video.mbus.width = wsize->width;
    sensor->video.mbus.height = wsize->height;
    sensor->video.mbus.code = wsize->mbus_code;
    sensor->video.mbus.field = V4L2_FIELD_NONE;
    sensor->video.mbus.colorspace = wsize->colorspace;
    sensor->video.fps = wsize->fps;

#ifdef SENSOR_RISCV_HF_MODE
    ret = gc2053_write_array(sd, wsize->regs);
    if (ret)
        return ret;
#endif

#ifndef SENSOR_WITHOUT_INIT
    ret = gc2053_write_array(sd, wsize->regs);
    if (ret)
        return ret;
#endif
    if(!riscv_is_pass){
        ret = gc2053_write_array(sd, wsize->regs);
        if (ret)
            return ret;
    }
```

```
char * get_sensor_name(void)
{
    return "gc2053";
}

int get_sensor_i2c_addr(void)
{
    return 0x37;
}

int get_sensor_width()
{
    return gc2053_win_sizes->width;
}

int get_sensor_height()
{
    return gc2053_win_sizes->height;
}
```

去掉所有带有 **module** 的内容（图中未全部列出，查找删除即可）

```
static int reset_gpio = GPIO_PA(18);
//module_param(reset_gpio, int, S_IRUGO);
//MODULE_PARM_DESC(reset_gpio, "Reset GPIO NUM");

static int pwdn_gpio = -1;
//module_param(pwdn_gpio, int, S_IRUGO);
//MODULE_PARM_DESC(pwdn_gpio, "Power down GPIO NUM");

static int sensor_gpio_func = DVP_PA_LOW_10BIT;
//module_param(sensor_gpio_func, int, S_IRUGO);
//MODULE_PARM_DESC(sensor_gpio_func, "Sensor GPIO function");

static int data_interface = TX_SENSOR_DATA_INTERFACE_DVP;
//module_param(data_interface, int, S_IRUGO);
//MODULE_PARM_DESC(data_interface, "Sensor Date interface");

static int sensor_max_fps = TX_SENSOR_MAX_FPS_25;
//module_param(sensor_max_fps, int, S_IRUGO);
//MODULE_PARM_DESC(sensor_max_fps, "Sensor Max Fps set interface");
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.2.2 编译 sensor 驱动

进入“os/drivers/sensor-t23/zeratul”。修改 Makefile 文件使其能够找到 kernle 和 isp 的 include。这里 Makefile 文件可以使用已经提供的 zeratul 平台其他 sensor 源码中的 Makefile，确定好路径修改 sensor 名称即可。

在编译 sensor 驱动之前要注意先编译一遍 kernel。

输入命令 `make` 编译 sensor 驱动。

输入命令 `make release` 完成 sensor 驱动的添加。

2.3 修改编译脚本

添加一款新的 sensor，必须也要改/build/env_setup_zeratul.sh,使其能够正确检索到我们添加的 sensor 文件。

还是以 gc2053 为例，打开 env_setup_zeratul.sh，找到下面 part1 和 part2 部分，添加如下加粗字体内容

Part 1:

```
function print_sensors()
    echo "  [1] gc2053"
    echo "  [2] os02k10"
    echo "  [3] jxf37"
    echo "  [4] gc2053"
    echo
 }
```

Part 2:

```
if [ -f build/env_setup_zeratul.sh ]; then
        SENSOR=os02k10
    elif [ "${SENSOR}" = "3" ] ; then
        SENSOR=jxf37
    elif [ "${SENSOR}" = "4" ] ; then
        SENSOR=gc2053
    else
        SENSOR=jxf37 ##default
    fi
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

上述添加完成后，重新 source 环境。终端输入 export 命令，可以看到如下打印，找到“ZRT_ENV_SENSOR”。即为环境<sensor>，这里<sensor>是必不可少的：

```
declare -x ZRT_ENV_BUILD_DIR="/home/kwang/work/zeratul/release/build"
declare -x ZRT_ENV_BUILD_JOBS="1"
declare -x ZRT_ENV_BUILD_TYPE="release"
declare -x ZRT_ENV_DRIVERS_DIR="/home/kwang/work/zeratul/os/drivers"
declare -x ZRT_ENV_FW_VERSION="\"ZRT_release_202008301244_kwang_ubuntu-64;[VERSION];ver=CAMERA_SOC_T31Z_gc2053_4.7.2_V;\""
declare -x ZRT_ENV_KERNEL_RELEASE="3.10.14-Archon"
declare -x ZRT_ENV_OS_DIR="/home/kwang/work/zeratul/os"
declare -x ZRT_ENV_OUT_CAMERA_BOOT_FILE_PATH="/home/kwang/work/zeratul/out/camera/boot.bin"
declare -x ZRT_ENV_OUT_CAMERA_DIR="/home/kwang/work/zeratul/out/camera"
declare -x ZRT_ENV_OUT_CAMERA_KERNEL_FILE_PATH="/home/kwang/work/zeratul/out/camera/uImage.zrt"
declare -x ZRT_ENV_OUT_CAMERA_RECOVERY_FILE_PATH="/home/kwang/work/zeratul/out/camera/recovery.bin"
declare -x ZRT_ENV_OUT_CAMERA_ROOTFS_FILE_PATH="/home/kwang/work/zeratul/out/camera/rootfs_camera.cpio.lzo"
declare -x ZRT_ENV_OUT_CAMERA_SYSTEM_FILE_PATH="/home/kwang/work/zeratul/out/camera/system.bin"
declare -x ZRT_ENV_OUT_CAMERA_TAG_FILE_PATH="/home/kwang/work/zeratul/out/camera/tag.bin"
declare -x ZRT_ENV_OUT_DIR="/home/kwang/work/zeratul/out"
declare -x ZRT_ENV_SENSOR="gc2053"
declare -x ZRT_ENV_SOC_TYPE="SOC_T31Z"
declare -x ZRT_ENV_TOOLS_DIR="/home/kwang/work/zeratul/release/tools"
declare -x ZRT_ENV_TOOL_CHAIN="4.7.2"
declare -x ZRT_ENV_TOP_DIR="/home/kwang/work/zeratul"
declare -x ZRT_ENV_TRANSFER_MODE="SPI"
declare -x ZRT_ENV_UPDATE_VERSION="CAMERA_SOC_T31Z_gc2053_SPI__4.7.2_V"
declare -x ZRT_ENV_USERLAND_DIR="/home/kwang/work/zeratul/release/userland"
```

2.4 添加 Sensor 所需 TAG 文件

进入”tools/make_tag”,找到如下目录：

```
riscv_fw
sensor_init_setting
sensor_calibration_bin
sensor_start_ae_table
```

<1>、Sensor riscv 固件：命名规则为 riscv_fw_<sensor>.bin，拷贝到 riscv_fw 目录下。

<2>、Sensor 初始化配置：命名规则为<sensor>_<txx>_init.c 拷贝到 sensor_init_setting 目录下，并且在 Makefile 中添加 sensor 编译。

<3>、Sensor 效果文件：命名规则为<sensor>_<txx>.bin，将其拷贝到 sensor_calibration_bin 目录下。

<4>、Sensor libz 文件：命名规则为 libz_<sensor>.c,将其拷贝到 sensor_start_ae_table 目录下。

其中以上文件添加方法如下：

<1> 不是必要项，可以暂时不添加。

<2> 是必添加项。

<3> 可以由其他的 sensor 效果文件拷贝过来。例如 jxf37_t23.bin 拷贝成 gc2053_t23.bin

<4> 可以由其他的 sensor libz 文件拷贝过来，例如 libz_jxf37.c 拷贝成 libz_gc2053.c

由于低功耗平台会提前初始化 sensor，所以我们会在 tag 中放入要初始化的相关

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

信息，如 sensor I2C 地址，配置列表等。所以主要重点讲述如何添加<2>文件，以 gc2053 为例，：

Step 1、创建 gc2053_t23_init.c（我们需要的文件）

Step 2、对比已有的 gc2063_t23_init.c（已有的一款 sensor）结构。将 Sensor 驱动源码中部分代码添加到我们创建的 gc2053_t23_init.c（我们需要的文件）文件中。需要添加的内容如下图所示：

例如下图 1 是 gc2053_t23_init.c 中 15fps dvp 接口的配置、I2C 地址等内容，这里的内容可以在**驱动源码中找到（路径 os/drivers/sensor-t23/zeratul）**。

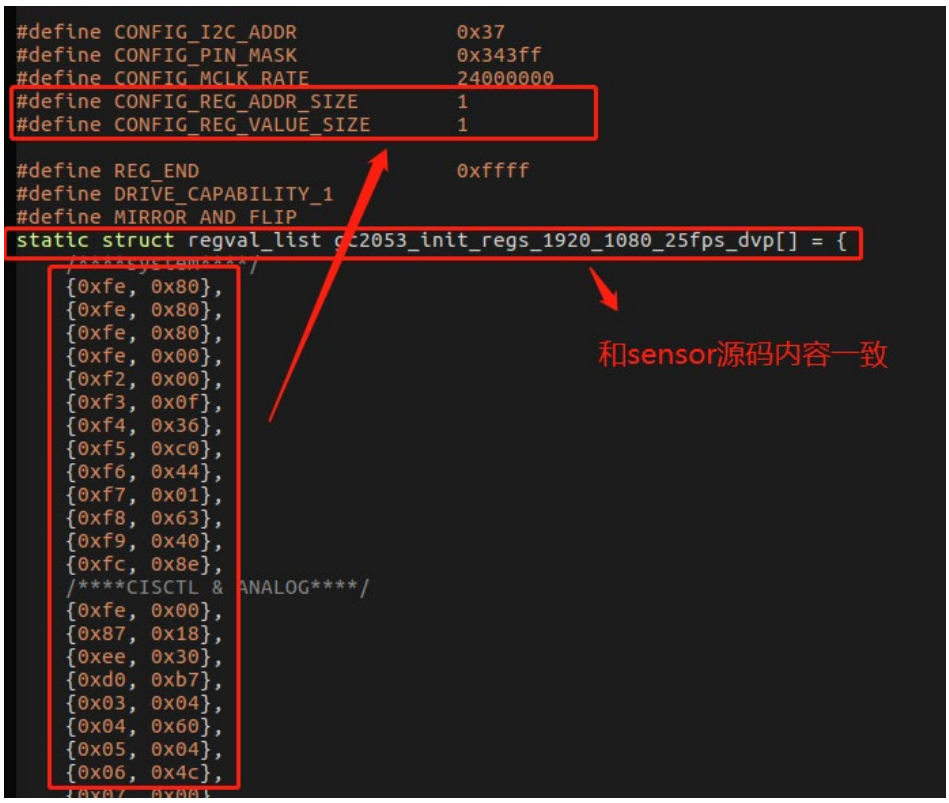
CONFIG_REG_ADDR_SIZE、CONFIG_REG_VALUE_SIZE 定义的大小根据 regval_list 里内容大小进行修改比如 0xfe 对应 1，0xffffe 对应 2。

CONFIG_I2C_ADDR 代表 I2C 地址，要根据 I2C 实际地址选择。（下图 3 位置）

图 2 是使用配置的内容。（驱动源码可以找到）

Step 3、`sensor_init_setting` 目录下仿照 Makefile 内容，添加 Makefile 内容使得 step 2 中创建的文件可以生成<sensor>_<txx>_init.bin 文件。

Step 4、输入命令 `make` 生成<sensor>_<txx>_init.bin 文件。



gc2053_t23_init.c 图 1

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```c
int main(int argc, char *argv[])
{
    if (argc == 1) {
        printf("usage: xxx_init <output file name>");
    }

    char *file_path = argv[1];
    int fd = open(file_path, O_CREAT | O_RDWR | O_TRUNC, 0644);
    if (fd < 0) {
        printf("create %s failed: %s\n", file_path, strerror(errno));
        return -1;
    }

    struct sensor_setting_info setting = {
        .head = 0,
        .len = 0,
        .crc = 0,
        .mclk = CONFIG_MCLK_RATE,
        .pin_mask = CONFIG_PIN_MASK,
        .i2c_addr = CONFIG_I2C_ADDR,
        .reg_addr_sise = CONFIG_REG_ADDR_SIZE,
        .reg_value_sise = CONFIG_REG_VALUE_SIZE,
    };

    int len;
    len = write(fd, &setting, sizeof(struct sensor_setting_info));
    len = write(fd, &gc2053_init_regs_1920_1080_15fps_dvp, sizeof(gc2053_init_regs_1920_1080_15fps_dvp));

    close(fd);

    if (len < 0) {
        printf("write error:%s\n", strerror(errno));

        return -1;
    }

    printf("generate sensor init setting file [%s] len=%d\n", file_path, len);

    return 0;
}
```

需要的配置

gc2053_t23_init.c 图 2

```c
struct tx_isp_sensor_attribute gc2053_attr={
    .name = "gc2053",
    .chip_id = 0x2053,
    .cbus_type = TX_SENSOR_CONTROL_INTERFACE_I2C,
    .cbus_mask = V4L2_SBUS_MASK_SAMPLE_8BITS | V4L2_SBUS_MASK_ADDR_8BITS,
    .cbus_device = 0x37,
    .dbus_type = TX_SENSOR_DATA_INTERFACE_DVP,
    .dvp = {
        .mode = SENSOR_DVP_HREF_MODE,
        .blanking = {
            .vblanking = 0,
            .hblanking = 0,
        },
        .dvp_hcomp_en = 0,
    },
    .data_type = TX_SENSOR_DATA_TYPE_LINEAR,
    .max_again = 453170,
    .max_again_short = 0,
    .max_dgain = 0,
    .min_integration_time = 4,
    .min_integration_time_short = 4,
```

源码 sensor 驱动 I2C 地址位置图 3

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3 sensor 问题调试说明

2.1 sensor 常见 FAQ

Q：ISVP 平台和 Zeratul 平台使用 Sensor 区别

A：如果使用的是常规平台（ISVP）
要加载如下驱动：
insmod *tx-isp-txx.ko* →ISP 驱动
insmod *sensor-txx.ko* →>Sensor 驱动
要注意的是 Sensor 要确定相应的 ISP 驱动，如 T23 Sensor 要对应 T23 ISP 驱动。
由于驱动依赖内核，编译驱动前要把 kernel 编译好。

A：如果使用的是电池平台（Zeratul）
不同于 isvp。zeratul 平台的 sensor 相关驱动是直接编译进内核的，不需要手动加载相关 sensor 驱动。当选择相应的 sensor 型号，直接编译进内核

Q：运行 sample 程序发现 err:video drop、chip found @0x**（i2c0）is not an ** chip 等错误。

A：video drop 说明 sensor 没有正常的出流。chip found @0x**（i2c0）is not an ** chip 说明了在对 Sensor 进行探测失败。具体排查参考小节：Sensor 硬件问题排查、Sensor 软件问题排查。

Zeratul 平台进一步排查：
打开相应的 sensor 驱动源码，注释掉宏 SENSOR_WITHOUT_INIT 。再次编译驱动。正常启动后，输入 dmesg 命令查看是否有报 I2C 错误。如果有错误打印，说明 I2C 未能通信成功，需要检查一下硬件问题。

Q：运行 sample 程序发现：IMP_Encoder_PollingStream(0) timeout。但是没有 err:video drop

A：由于上层与驱动层需要密切配合，所以，有些配置信息需要提前配置，如 sensor 的宽高信息和所配置的 video buffer 个数。这些信息将被写入到 TAG 分区的环境变量中。并且这些信息需要和主程序的配置完全一致。可尝试在设备串口里，利用 tag_env_info 工具写入

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

`tag_env_info --set HW init_vw 1536` // sensor 的宽

`tag_env_info --set HW init_vh 1376` // sensor 的高

`tag_env_info --set HW nrvbs 2` // video buffer 个数，默认设置为 2，内存够用的话，建议设置为 3 到 4 个

Q：运行 sample 程序发现：主码流帧率减半

找到 cmdline 中 nrvbs 数值和 sample 设置数据是否一致。

`tag_env_info --set HW nrvbs 2`

命令设置 nrvbs

Q: 图像显示不正常

A：Sensor Data 线对齐方式设置错误。需要根据 SCH 的设计，insmod 时加入参数。

例如：**insmod** *sensor_gc2053.ko sensor_gpio_func=1*，其中参数值的意义为：

- 0: GPIO-A 低 10bit
- 1: GPIO-A 高 10bit
- 2: GPIO-A12bit
- 3: GPIO-A 低 8bit
- 4: GPIO-A 高 8bit

Q: 图像显示偏绿

A：图像偏绿的原因可能是没有在文件系统中放置 bin 文件(/etc/sensor/xxx.bin)，在此情况下会有内核打印：ISP: open %s file for isp calibrate read failed。或者 Sensor bin 文件与 ISP driver 不匹配，会出现如下打印：

```
#################################################################
  #### The version of ov9750.bin doesn't match with driver! ####
  #### The version of ov9750.bin is xxx, the driver is yyy ####
#################################################################
```

Q: 图像显示花屏

A：图像花屏的问题可能是 Sensor DVP 线(pclk, sync)的驱动能力不足

Q：图像条纹

A：按照以下几点排查：

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

- 检查 senosr 初始化 setting 是否正确
- 检查 sensor 板温度是否正常
- senosr 的 AVDD 以及 DVDD 是否正确，AGAIN 是否设置过大

Q: 如何修改 Sensor 默认帧率

A: 开启 ISP 数据流的操作流程如下：

```
IMP_ISP_Open();
IMP_ISP_AddSensor();
IMP_ISP_EnableSensor();
IMP_ISP_EnableTuning();
/**
 * 如果想改变 ISP 原始输出帧率，这时请调用
IMP_ISP_Tuning_SetSensorFPS();
 * 如果不改变 ISP 原始输出帧率，不进行操作。
 * /
FrameSourceInit()
IMP_FrameSource_CreateChn()
IMP_FrameSource_EnableChn()
```

Q: 关于设置帧率接口，帧率如何定义？Sensor FPS 和 Encoder FPS 有什么区别或联系？

A: ISVP 中的帧率为分数形式，由两个数据组成：Num 和 Den，其中 Num 是分子（被除数），Den 是分母（除数），因此，FPS=Num/Den，例如：对于 FPS=25，Num=25，Den=1，对于 FPS=12.5，Num=25，Den=2。此外，应确保 Num 和 Den 的最大公约数为 1。

关于 Sensor FPS 和 Encoder FPS 有什么区别或联系：

Sensor 的 FPS 为系统的输入帧率，通过配置 Sensor 的 VTS 参数实现。一般在夜视时需要降低 Sensor 帧率，以获得在较低 Gain 下足够的曝光。

Encoder（H264）的 FPS 为输出的 H264 视频流帧率，降低 Encoder FPS 可有效降低码率。

Encoder FPS 通过丢帧实现，因此 Encoder FPS <= Sensor FPS，值得注意的是，当改变 Sensor FPS 时，Encoder FPS 会被同时自动改变，而改变 Encoder FPS 时，Sensor FPS 不会自动改变。

改变 Encoder 的帧率也可以实现两路编码不同帧率。

Q：error lfb open faild

错误原因：isp 没接收到 sensor 数据，比如下图

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
error: lfb open failed!
CPU: 0 PID: 1 Comm: swapper Not tainted 3.10.14-Archon #2
Stack : 00000000 8003e9a8 00000000 8003e1e8 00000000 00000000 00000000 00000000
   00000000 00000000 804b2dea 00000039 8202c1d8 00000b00 00000000 00000000
   00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
   00000000 00000000 00000000 00000000 00000000 8202ba00 8202bafc 803d47d4
   8041e1a7 800401d4 804b3cc0 803d47d4 00000000 00000001 8202c1d8 8202ba70
   ...
Call Trace:
[<8002195c>] show_stack+0x64/0x7c
[<80374f28>] dump_stack+0x20/0x2c
[<8035fa48>] isp_printf+0x88/0x90
[<80347c9c>] ispcore_pad_event_handle+0xeac/0xffc
[<803614f0>] ncu_pad_event_handle+0x740/0x948
[<803644f0>] mscaler_video_s_stream+0x42c/0x548
[<8033415c>] tx_isp_video_link_stream+0x70/0x13c
[<80336860>] start_tx_isp_device+0x19c/0x280
[<80336d88>] tx_isp_probe+0x444/0x474
[<801d8c04>] driver_probe_device+0x84/0x234
[<801d8e70>] __driver_attach+0xbc/0xc4
[<801d6c98>] bus_for_each_dev+0x58/0xa4
[<801d8704>] driver_attach+0x20/0x2c
[<801d8208>] bus_add_driver+0x110/0x274
[<801d9258>] driver_register+0x90/0x1c4
[<801da388>] platform_driver_register+0x58/0x64
```

A:。检查 MIPI 的时钟和数据的波形，检查 MIPI 接口原理图是否和 PCB 图画的一致。

2.2 sensor 不出图像排查

2.2.1、Sensor 不出图像硬件问题排查

●检测 Sensor 相关供电是否正常，是否达到厂家要求。这里可以看 Sensor 的技术手册。

●I2C 引脚是否是 I2C0，引脚对应是否正常。必要时可以抓取波形看是否有实际数据传输。

●MCLK 输出是否正常，一般来说是 24MHZ。

●确认使用 MIPI 接口或 DVP 接口，以及接口连接正常。要和软件 Sensor 驱动源码中使用的接口配置定义一致。

●确认 Sensor 的 RESET 和 PWDN 引脚是高电平有效还是低电平有效，尤其 PWDN、RESET 引脚是可以控制的。具体内容要查看 Sensor 手册，比如 JXF23，PWDN 引脚高电平有效，需要对其引脚进行下拉。RESET 低电平时复位，要对其上拉

●检查硬件的连接是否良好，可以更换排线或者 sensor 板试一下 。

2.2.2 Sensor 不出图像软件问题排查

●如果使用 sample 程序。sample-common.h 的 sensor 对应的 I2C 地址、分辨率是否和驱动中定义一致。

●如果使用 carrier-server 程序。carrier-server 是否添加了使用的 sensor 型号。板子是否能 ping 通主机。

●分辨率、裁剪、缩放等配置是否正确。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.